ECE 2564
Fall 2025
Homework 2

Please sign (You can type your name in place of signature.)

"I have neither given nor received unauthorized assistance on this assignment."

Please read carefully General instruction

- Accept the following GitHub assignment and create a new repository. https://classroom.github.com/a/PyDTWreT

- Import the repository to CCS. This will create ***three*** projects in your CCS. Use this tutorial if you need: https://sites.google.com/vt.edu/introduction-to-embeddedsystem/tutorials/github-integration/assignments/github-desktop-method/importing-assignments

- Solve problems 1 through 3 below. There are many steps in this assignment. However, only some steps require you to create content for your homework solution. Those parts are formatted in **bold**. You can create your solution using any document editor you wish.

- There are two parts to your submission:
    - Submit your updated code to GitHub using the **below instructions by the deadline**: https://sites.google.com/vt.edu/introduction-to-embeddedsystem/tutorials/github-integration/assignments/github-desktop-method/submitting-assignments

- Turn in the answer document as a PDF on Canvas **by the deadline**.

Problem 1 - 25 points

In this problem, we will practice debugging using CCS with a simple arithmetic operation example. The goal of this project is to multiply two integers and get a correct answer.

A. Make sure the F25_HW2_P1 project is “Active”, then “Build” the project by pressing the hammer button.
B. Start debugging by pressing the bug button.
C. Read the whole program including the comment(s). The program is very simple. It multiplies a and b and stores the result in c. It repeats it for x and y.
D. Make sure the “Variables/Expressions/Registers” window is open. If not, open it.
E. Click on either of “Step Into” or “Step Over” icons four times. Watch the variables as you step. After four steps, the blue arrow on the sidebar that shows which line is going to run next should be at line 9. **Make a screenshot and add it to your homework solution.** Use a calculator or manually multiply a and b. Does the c in your debugger match the answer you expect? **Describe whether an error exists or not and if yes, explain why.**
F. Click on either of the “Step Into” or “Step Over” icons three more times. Watch the variables as you step. **After the new three steps, take a screenshot and add it to your homework solution.** Use a calculator or manually multiply x and y. Does the z in your debugger match the answer you expect? **Describe whether an error exists or not and if yes, explain why.**
G. Fix the code such that both multiplications are done correctly **without using floating-point data types**. You are not allowed to change the type of any of the variables except for variables c or z. **Copy paste the correct code and include it in your solution. Explain in a short paragraph why the new code fixes the error. Recompile your new code and debug it. Take a screenshot that shows c and z are both correct.**

Problem 2 - 15 points

In this problem, we will practice debugging using CCS with an array example. The goal of this project is to copy an array into another one with a reversed order.

A. Make sure the F25_HW2_P2 project is “Active”, then “Build” the project.
B. Start debugging.

C. Read the whole program including the comment(s). Do you expect that program can safely copy the array and set the flag “done” to 1? (No need to include this answer in your solution.)
D. Make sure the “variables” tab is open and you can see the variables. Click on the two arrays “source” and “destination” that look like folders to open them. **Step until line 8 is highlighted and take a screenshot and include it in your solution.**
E. Step several times until you copy the entire array from source to destination. **Take another screenshot and include it in your solution.**
F. Continue to step. What is going on? Why doesn’t this loop end? **Fix the code such that the loop ends when the entire array is copied. Although there are a variety of ways to fix this problem, we are interested in fixing this in one way only: You need to change the type of variable i to achieve this. Once you fix it, compile and run the code again and make sure it is working properly. Copy and paste the correct answer in your solution file. In a short paragraph explain why the original code had a bug and how you have fixed it.**

Problem 3 - 60 points

In this problem, we will learn about strings and ASCII code and write a small function.

A. Make sure the F25_HW2_P3 project is “Active”, then “Build” the project.
B. Start debugging by pressing “Step Over”. As you debug, r**ead all the code lines including comments**. Make sure the “variables” window is open.
C. The first part of the code shows how you can build a string by manually filling it as an array of characters. Read the comments as you step over until the blue arrow passes line 52. By this time, you should have learned about NULL character in the string and its role. **Take a screenshot that shows the variables window and attach it to your solution. Also, take a photo of your kit and crop the image to only show the screen of the kit.**

D. **What would have been the output on the display if the code looked like this at lines 43-49?**

```
// Let's manually create a string that represents a number
unsigned char string1[6] = {1, 1, 1, 1, 1, 1};
string1[0] = '2';
string1[1] = NULL;
string1[2] = '6';
string1[3] = '4';
string1[4] = NULL;  // the NULL or terminating character. This indicates the
                    // end of the string
string1[5] = '5';
```

E. Another way to build a string is to use the snprintf library function. You can learn more about it: https://www.cplusplus.com/reference/cstdio/snprintf/
Step over until the arrow is passes line 73. Observe how it works.

F. Read the comment block between line 75 and 80.

G. Another way to build a string is use strncpy library function. You can learn more about it here: https://www.cplusplus.com/reference/cstring/strncpy/

H. Step Over until the arrow is at line 89. Make sure the Variables view is open in your debugger. Click on the string3 folder in the Variables view. Step over line 89 and observe the change in string3 content. **Take a screenshot and attach here. Circle or highlight the screenshot to show whether the string3 is NULL terminated or not.**

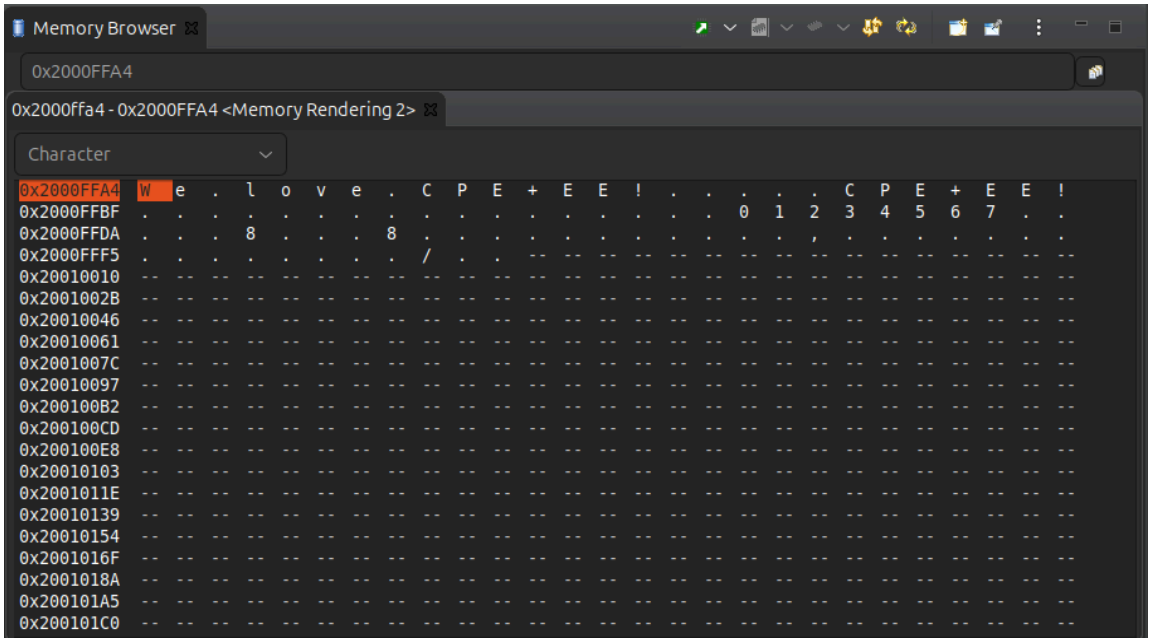
I. Step over line 93. Take a look at your board's display. Our hope was to see the word "CPE+EE!" on the screen. **Is this what you see? Take a photo and attach it here.**

J. Use the hints at line 94 to 97 to fix the above issue. You need to stop the debugging by pressing on the red square on top of the display. Fix your code back in the "edit" mode. If you need to add a new line of code, use line 91. Do not add a new line to the program as the rest of this homework uses line numbers to guide you through the steps and we do not want those numbers to not match your program. After fixing your code press the hammer. Make sure you do not have any errors or new warnings. Now, press the bug button. This should flash your code and allow you to debug again. Instead of stepping over all the way to line 89, create a breakpoint at line 89 by double-clicking on the narrow blue side-bar right beside the number 89. You should see a circle appear where you click. Now, press the green triangle play button on the top of the debug window. This should redo everything we did earlier and get you to line 89. Step over line 89 (and possibly 91, if you added it) and watch the variable view. **Is your string3 now NULL-terminated? Copy-paste or take image snippet of your new lines 89-91 and attach here.**

K. Step over line 93. You should see the desired behavior now if you have properly fixed your code above. **Take a screenshot of the screen with the fixed behavior.**

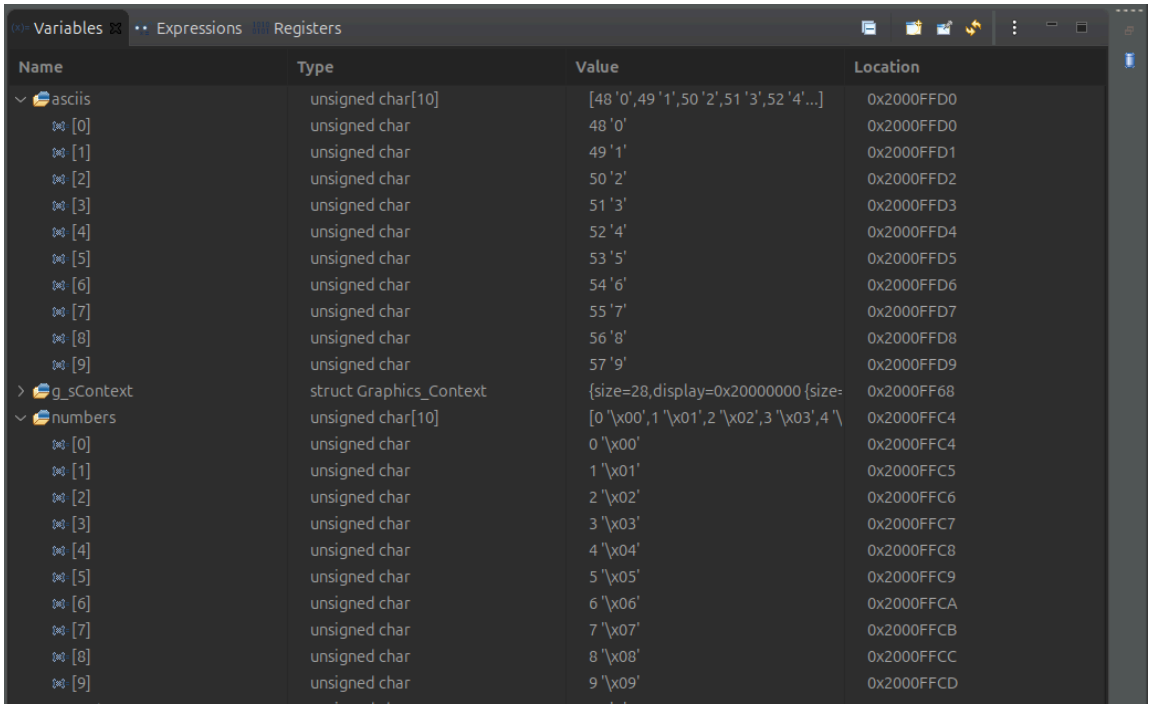
L. Another function that helps with string manipulation is strncat. You can learn more about it here: https://www.cplusplus.com/reference/cstring/strncat/

M. Read the comments and step over while observing string4 until you pass line 112. Click on “View” and then choose “Memory Browser”. This should add the memory browser to your view:



N. Type the address of string4 (for me, it is 0x2000FFA4) where you see the search bar under Memory browser. From the drop-down menu below it, choose “Character”. You should see the message we created in string4 in the memory. **Take a screenshot of the memory browser and attach it here.**

O. Step over until you pass line 124. In the Variables window, click on the small arrow for folders that hold the arrays, ***numbers*** and ***asciis***. This will result in all their elements to show. You will see something like this:



As you can see each character is shown both as a number and as its ASCII form in the variables view for both arrays. The ASCII code is shown first, followed by how it "looks". For example, code 48 represents '0' and code 49 represents '1'. *It looks like if you have a number between 0 and 9 and you add 48 to it, you get its ASCII equivalent.* You can learn more about ASCII characters here.

P. Stop the debugger using the square red button on the top.

Q. In the editor mode, continue reading the rest of the program. You need to develop a function that reads months, days, and years as an input number and creates a string that shows the date. For example, if the month is 1, the day is 1 and the year is 2025, the string will be "01/01/2025". The string needs 10 characters plus a NULL character. So, we have declared a string with 11-character's worth space. Carefully read the comments above the function.

   You can have two totally different approaches to write this function:

   1) Use your knowledge of how numbers and ASCII code are related to each other to write this function.
   2) Use snprintf. There is a way to make snprintf print a predefined number of characters such as 00 instead of 0.

   Regardless of your method, *make sure your string is NULL-terminated.*

R. Build your code and start debugging again. To fast forward to the part where the `make_date_string(10, 31, 2032, string5)` function is, you can double click on the

line number 132 to create a breakpoint and use the “Play” triangle green button instead of step over.(Remove the old breakpoint by double-clicking on it).

S. If you are interested to see if the function you wrote works, you need to use the “Step Into”, instead of “Step Over” to go inside the function. You can use “Step Return” to get out of a function or simply continue to step over until you reach the end of the function. It will automatically take you back to the caller function.

T. Step Over all the remaining lines. If your function is written correctly, you will see a screen similar to this on your kit:

10/31/2032
05/06/1905
08/02/2022

U. Copy and paste the code you have developed for the function in your solution file.

V. Submit your code to GitHub. Use a summary message that contains the words similar to “final submission”. Attach a screenshot from the GitHub website that shows your submission status. It will look like the below image.

